

# DAAM Scout – Phase 1 Plan

## Conference Finder, Fit Matcher, and who should go

**GOAL:** Web app (“Scout”) that finds AI Conferences, tells the team how well each one matches the product’s messaging, and suggests which teammate should go. Runs without any internal data feeds. No Salesforce, no sales data, no email or social data just yet. This is mostly external OSINT data, and messaging and positioning data to determine semantically if we should go to X or Y conference.

**USE CASE:** To determine with a deeper understanding if we should go to X or Y conference, the requirements of these conferences, and to determine if we have SME’s on the DAAM team that can attend and support these events.

## INTENDED FEATURES:

- **Find Conferences:**
  - Crawls the web for upcoming AI events and conferences and keeps a clean, deduplicated list. The list will include name, dates, locations, topics, audience (related to Red Hat list of Audiences), CFP window, and estimated cost.
- **Matches Fit:**
  - Compares each conference against the products current messaging and positioning and Red Hat AI’s four pillar strategy, and will return a fit rating and a short written reason to attend or apply.
- **Picks the right teammate or SME:**
  - Suggests who on the DAAM team (or Non-DAAM SME if needed) should attend these conferences based on the topics, audience, and the team profiles entered into the app.
- **Team Review:**
  - Dashboard with a ranked conference list, a detailed view per conference, and approve or reject actions.

## WHAT THE TEAM MANUALLY ENTERS:

- **Product Messaging & Positioning**
  - Short paragraphs the team writes (elevator pitch, target personas, key themes, talking points)
  - Also uploaded via PDF to be ingested into a RAG based knowledge graph.
- **Audience Profiles (Red Hat Audience Profiles)**
  - Each Red Hat persona/ audience based on the Corporate Marketing and Sales definition of each.



- **DAAM SME's**
  - Area of expertise, topics they care about, audiences they speak to well, past conferences they have attended or talked at, where they are located etc....
- **Non-DAAM SME's**
  - Team name, Area of expertise, topics they care about, audiences they speak to well, past conferences they have attended or talked at, etc...
- **Conferences Attended:**
  - Can import via CSV or can enter in past conferences attended. (may be superseded/ removed by phase 2).

## **BUILD PLAN/ IMPLEMENTATION PLAN:**

### **1. Framework:**

- ROUTE 1 - Hybrid API Cloud via MaaS:
  - Fast API Backend
  - Postgres with pgVector
  - React Frontend (or NextJS).
  - OpenAI API - MaaS model serving infrastructure
    - Choose any model & number of models
    - Easier to maintain
- ROUTE 2 - Locally Hosted:
  - Flask Web App
  - PostGres/ PgVector
  - LlamaC++ as the backend inference engine
  - Single 9B Qwen Model (12-15GB VRAM)
    - Limited to vRAM and single model
- ROUTE 3 - Fully Cloud Hosted
  - Openshift Managed (need if many people are using at once).
    - NextJS node
    - PostGres Node
    - Nginx Node (load balancer)
    - vLLM node OR llm-d node for multi-agent
      - Have to manage vLLM or LLM-d node
      - Can choose any model

### **2. Data Schema:**

- Build data schema tables based on key trackable items. It is important to start building this first (PostGres).
- Input data -> Aggregated Data -> schema for collected data/ output data.



### 3. UI/ UX:

- Easy to navigate
- Dashboards are easily understandable, and show only key metrics
  - Maybe include a dropdown of dashboard views, so each individual user can filter out only what is important to them.
- Section to talk directly to the Agent about decisions, etc...
- Settings section to include any manual entry data points, or a page for manual data entry.
- Potentially show the knowledge graph of ingested data and the labeled and structured data to show the semantic similarity between data points (Like obsidian).

### 4. Data Scraper/ Web Scraper:

- a. Crawl4AI Open Source project for normal pages, potentially utilize OpenShell and playwright for Java-script heavy sites? Scheduled to run at a click of a button, or at a set cadence.
  - i. Raw Pages first, ingest second

### 5. Clean and Structure Data:

- a. RAG ingestion for PDF and Web Data/ OR JSON format for LLM.
  - i. Ingested into knowledge graph
- b. Validation & Confidence routing on data ingested
- c. Utilize a Knowledge graph OR potentially Page Index depending on the ingested data structure of the websites.
  - i. PgVector is a good vector DB to utilize.
- d. Reject and Flag failed ingested sites.
- e. Implement Ebbinghaus Decay to clean out old unused data
  - i. Data half-lives after X time of not being called, used, etc...
  - ii. Most prominently used in managing context windows.
- f. File Structure and dates on files ingested as well.
- g. STRICT Data rules on what you can upload. A User cannot upload anything and everything. Only documents in-which we need data from.
- h. Git Blame type of feature to have the ability to update content.

### 6. Fit Matcher:

- a. Algorithm to set up at specific levels of cosine similarity, like Top-K optimizations to ensure that the RAG pipeline is ingesting and giving good results based on similarity between an individual's expertise AND the conference they are attending AND the messaging and positioning of said conference.
  - i. Algorithm should follow something similar to this:
    - 1. Algorithm should use these dimensions:
      - a. Audience, Messaging & Positioning, Strategy, SME Expertise, SME Location



2. X Conference -> Does X semantically relate to Y messaging and positioning? -> Does X semantically relate to Four Pillar Strategy  
- > Does X relate to Z individual or SME
3. Each check can have exits for an action:
  - a. If it does not relate to messaging and positioning, then its not included in the list
  - b. If it does relates to messaging, but does not follow four pillars, then human review
  - c. If it follows messaging and positioning, and strategy pillars, but does not relate to any SME, then human review to find a SME to support.

## **PHASE 2:**

Phase 2 will expand upon the data schema and add dimensionality to the matching algorithm using data found in the Dataverse. Everything in Phase 1 should be set up as a foundation to support Phase 2 expansion.

## **SHORT-TERM & LONG TERM MAINTENANCE:**

Depending on the infrastructure and framework route taken, the best route would be to use a simple GitHub CI/CD Pipeline (opensource type application). Where maintainers can access source code, documentation, data schema setups, guardrails/ Jinja templates, etc... Otherwise, a dedicated team or person will need to continually support the efforts wherever the application lives.